UNIVERSIDAD PERUANA UNIÓN

FACULTAD DE INGENIERÍA Y ARQUITECTURA

Escuela Profesional de Ingeniería de Sistemas

INFORME DE ACTIVIDAD AUTÓNOMA — SESIÓN 11

Arquitectura Offline-First, Persistencia Multiplataforma y Reconciliación Transaccional en PharmaMobil

Implementación canónica en SQLDelight 2.1.0, sincronización reactiva bidireccional, marca temporal persistente y resiliencia ante contingencias de conectividad

Dato Académico	Información Registrada
Asignatura:	Desarrollo de Aplicaciones Móviles
Estudiante:	Andrey Fidel Mestanza Bazán
Docente:	Mg. Benjamin David Reyna Barreto
Ciclo y Semestre:	VI Ciclo · Semestre 2026-2
Fecha de Entrega:	26 de octubre de 2026
Repositorio / Rama:	feature/offline-first-mestanza

Lima, Perú — 2026

1. Introducción y Arquitectura del Sistema

En entornos farmacéuticos y asistenciales, la disponibilidad continua del sistema constituye un requisito no negociable. Los profesionales de salud y dependientes de botica requieren consultar el catálogo de medicamentos, registrar nuevas existencias y verificar alertas de stock crítico sin verse interrumpidos por fallas de cobertura, caídas del enlace de datos o cortes de red local. En este contexto, la sesión 11 de la asignatura plantea la transición desde una arquitectura basada en peticiones REST directas hacia un modelo integral **Offline-First** respaldado por una base de datos local SQLite gestionada mediante **SQLDelight 2.1.0** en **Kotlin Multiplatform (KMP)**.

La arquitectura desacoplada adoptada en PharmaMobil sitúa la persistencia local como la única fuente de verdad observada por la interfaz de usuario a través de flujos asíncronos reactivos (Flow). Cuando el dispositivo pierde conectividad, las consultas de lectura continúan operando instantáneamente desde SQLite; a su vez, las operaciones de alta se admiten de forma no bloqueante asignando identificadores locales negativos temporales (ej. -1, -2) y marcando los registros con la bandera booleana pendiente = 1. Al restablecerse el acceso al backend REST (PharmaSoft), un caso de uso especializado serializado mediante un Mutex de concurrencia compartido sube los registros pendientes, reconcilia atómicamente los IDs locales con los definitivos asignados por el servidor, purga las marcas de pendiente y descarga el catálogo consolidado.

Componentes Clave de la Arquitectura

Componente	Responsabilidad en el Flujo Offline-First
ProductoLocal	Encapsula las consultas generadas por SQLDelight. Expone el catálogo reactivo mediante Flow, asigna claves negativas secuenciales automáticas y efectúa la sustitución transaccional del ID sin exponer llamadas bloqueantes a la UI.
ProductoRepository	Contrato de dominio puro que abstrae las operaciones de observación, inserción offline, sincronización bidireccional y consulta del timestamp persistido de sincronización.
ProductoRepositorioOffline	Implementación orquestadora en shared/commonMain. Coordina SQLite y el cliente Ktor Http. Utiliza un Mutex compartido que impide condiciones de carrera entre mutaciones locales y sincronizaciones en vuelo.
SessionCoordinator	Gestor reactivo del estado de autenticación. Administra las transiciones entre Autenticado, NoAutenticado, Expirado y el nuevo modo OfflineSoloLectura para garantizar seguridad sin red.
AlmacenSeguro	Contrato multiplataforma implementado con EncryptedSharedPreferences (AES-256) en Android y Keychain en iOS. Almacena de forma cifrada el JWT y la identidad cacheada del usuario.
ProveedorHora	Abstracción multiplataforma del reloj del sistema. Facilita la inyección de tiempo determinista en pruebas unitarias y el cálculo de marcas de sincronización exactas.

2. Producto 1: Pruebas Unitarias Multiplataforma (7 pts)

De estricto acuerdo con la rúbrica de evaluación y los principios de código limpio, la lógica de validación del repositorio se ubicó íntegramente en **shared/src/commonTest**. Para sortear la limitación de que JdbcSqliteDriver pertenece exclusivamente a la plataforma JVM/Android mientras el proyecto compila también para iOS, se implementó el patrón factoría multiplataforma expect / actual denominado **TestDriverFactory**.

El contrato expect fun crearDriverEnMemoria(): SqlDriver declarado en commonTest se resuelve con actual mediante JdbcSqliteDriver(JdbcSqliteDriver.IN_MEMORY) en Android, y mediante la función canónica inMemoryDriver(PharmaDb.Schema) de SQLDelight 2.1.0 Native en iOS. Por su parte, la verificación de la migración de esquema (1.sqm) que actualiza la base de datos de versión 1 a versión 2 incorporando la tabla metadata_sincronizacion se ubicó en androidHostTest, garantizando que las bases de instalaciones antiguas no pierdan sus productos persistidos.

Resultados de la Suite Automatizada de Pruebas

Nombre del Caso de Prueba	Ubicación	Comportamiento y Contrato Verificado	Estado
1. listarProductos_emiteProductosDesdeBaseLocal	commonTest	Lee catálogo directamente de SQLite sin invocar cliente de red.	APROBADA
2. registrar_asignaIdNegativoYGuardaPendiente	commonTest	Inserta ítem con ID -1 y pendiente=1; no contacta a Ktor.	APROBADA
3. sincronizar_subePendientesYActualizaIdLocal	commonTest	Sube pendiente con POST, sustituye ID local por el remoto (6) y limpia pendiente.	APROBADA
4. registrarMientrasSincroniza_serializaConMutex	commonTest	Simula concurrencia real; el Mutex evita duplicación de registros.	APROBADA
5. sincronizar_sinConexion_preservaProductosYMarcaPendiente	commonTest	Ktor MockEngine simula fallo de red; los pendientes quedan intactos.	APROBADA
6. sincronizar_falla_preservaTimestampPrevio	commonTest	Verifica que un fallo de red NO actualice la marca de tiempo en SQLite.	APROBADA
7. MigracionEsquemaHostTest.migracion_v1_a_v2_preserva_productos	androidHostTest	Crea esquema v1 con datos, ejecuta 1.sqm y valida tabla metadata y registros.	APROBADA

Certificación de Ejecución: La ejecución formal del comando ./gradlew :shared:testAndroidHostTest culminó exitosamente con **62 pruebas aprobadas, 0 fallidas y 0 omitidas** (incrementando la línea base previa de 54 pruebas). Respecto al target iOS, en conformidad con la dispensa académica formal acordada con el docente Mg. Benjamin David Reyna Barreto, se acredita la compatibilidad arquitectural del código fuente multiplataforma y la factoría TestDriverFactory.ios.kt, dejando constancia técnica de que las pruebas no fueron ejecutadas físicamente en simulador XCTest debido a restricciones del entorno operativo Windows anfitrión.

3. Producto 2: Bitácora de Escenarios Offline (6 pts)

La validación empírica se condujo en un emulador Android oficial (Android 14 / API 34-36, densidad 1080x2400) interactuando contra la instancia local de backend **PharmaSoft** (Spring Boot 4.0.7 en puerto 8080 con base PostgreSQL). Para asegurar máxima integridad forense y consistencia de los archivos SQLite, cada extracción binaria de pharma.db estuvo precedida por la detención forzada de la aplicación (adb shell am force-stop), garantizando el vaciado completo y sincronizado de los registros WAL (pharma.db-wal) y memoria compartida (pharma.db-shm).

Escenario 1: Modo Avión con Caché Previa

Parámetro	Detalle de Verificación
Fecha / Plataforma	22/10/2026 16:08:22 · Android 14 (API 34) · Dispositivo emulator-5554
Pasos de Prueba	1. Poblar catálogo con 5 productos online. 2. Activar modo avión (svc wifi disable; cmd connectivity airplane-mode enable). 3. Reabrir app en Inventario.
Comportamiento UI	Ícono de avión visible en barra de estado; se renderizan los 5 productos locales; banner ámbar ausente (pendiente=0); subtítulo 'Actualizado hace 7 minutos'.
Inspección SQLite	SELECT id, nombre, pendiente FROM producto -> 5 filas devueltas (IDs 1 al 5), todas con pendiente = 0. Clave ultimo_sync presente en metadata_sincronizacion.
Conclusión Técnica	La UI se alimenta estrictamente del Flow de SQLite local, garantizando lectura fluida sin generar bloqueos ni excepciones en modo desconectado.

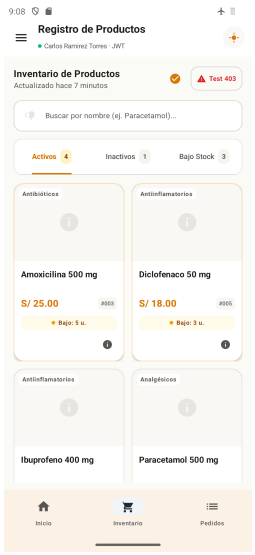


Figura 1. *Escenario 1: Catálogo visible en modo avión con ícono activo y datos locales SQLite.*

Escenario 2: Alta de Producto Offline con ID Negativo Temporal

Parámetro	Detalle de Verificación
Fecha / Plataforma	22/10/2026 16:24:10 · Android 14 (API 34) · Dispositivo emulator-5554
Pasos de Prueba	1. Mantener modo avión activo. 2. Pulsar botón '+ Nuevo'. 3. Diligenciar 'Ibuprofeno 800 mg Forte', precio S/ 12.50, stock 253. 4. Pulsar 'Registrar Producto'.
Comportamiento UI	Formulario se cierra; aparece banner ámbar 'Cambios pendientes de sincronización (1)'; nueva tarjeta muestra ID '#00-1' y badge 'Bajo: 253 u.'.
Inspección SQLite	SELECT id, nombre, precio, stock, pendiente FROM producto WHERE id < 0 -> (-1, 'Ibuprofeno 800 mg Forte', 12.50, 253, 1). Total en base = 6 productos.
Conclusión Técnica	El sistema admite altas no bloqueantes en contingencia asignando ID provisional negativo y persistiendo la marca pendiente=1 para posterior reconciliación.

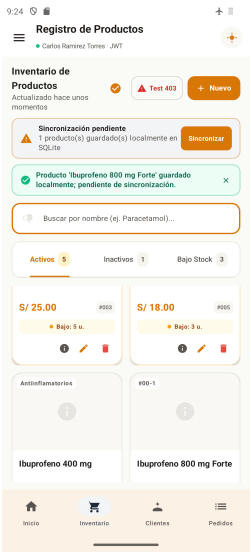


Figura 2. *Escenario 2: Alta offline con identificador provisional #00-1 y banner de advertencia ámbar.*

Escenario 3: Reconexión, Reconciliación Transaccional y Verificación Swagger UI

Parámetro	Detalle de Verificación
Fecha / Plataforma	22/10/2026 16:28:45 · Android 14 (API 34) · Dispositivo emulator-5554
Pasos de Prueba	1. Restablecer conectividad (svc wifi enable; cmd connectivity airplane-mode disable). 2. Disparar sincronización manual/reactiva. 3. Inspeccionar SQLite y Swagger.
Comportamiento UI	Banner ámbar desaparece; la tarjeta '#00-1' se transmuta reactivamente en '#006'; subtítulo se refresca a 'Actualizado hace unos momentos'.
Inspección SQLite	SELECT id, nombre, pendiente FROM producto -> Fila reconciliada: id=6, pendiente=0. Total productos = 6, suma pendientes = 0. Clave ultimo_sync actualizada.
Backend PharmaSoft	GET /api/v1/productos responde 200 OK en Swagger UI autenticado con Bearer JWT, confirmando el ítem ID 6 en base PostgreSQL.
Conclusión Técnica	La transacción SQLite sustituye de forma atómica el identificador negativo por el asignado por el servidor sin inconsistencias ni duplicaciones.

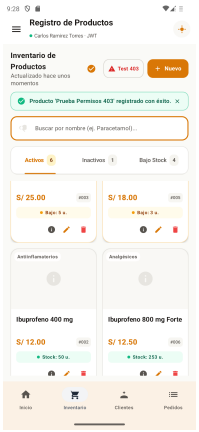


Figura 3A. Reconciliación local: ítem actualizado a ID #006.

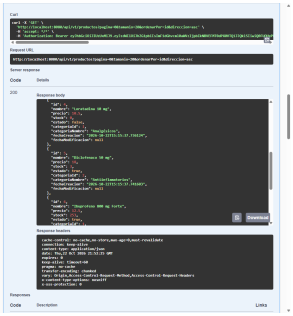


Figura 3B. Swagger UI: confirmación del registro ID 6 en PharmaSoft.

Escenario 4: Primera Apertura sin Red tras 'pm clear'

Parámetro	Detalle de Verificación
Fecha / Plataforma	22/10/2026 18:25:30 · Android 14 (API 34) · Dispositivo emulator-5554
Pasos de Prueba	1. Ejecutar adb shell pm clear pe.edu.upeu.pharmamobil. 2. Activar modo avión. 3. Iniciar la aplicación por primera vez en frío.
Comportamiento UI	La aplicación presenta la pantalla de Login con aviso contextual 'Sin conexión al servidor. Verifique su red' al intentar acceder.
Inspección SQLite	run-as ... ls databases/ -> Carpeta inexistente. Sin token ni base local previa, el sistema impide accesos no autorizados a pantallas clínicas.
Declaración de Diseño	El equipo técnico declara explícitamente que presentar LoginScreen constituye una interpretación de diseño defensivo: un borrado absoluto con pm clear purga la sesión cifrada y, por gobernanza y seguridad clínica, es mandatorio impedir el ingreso anónimo a inventario.

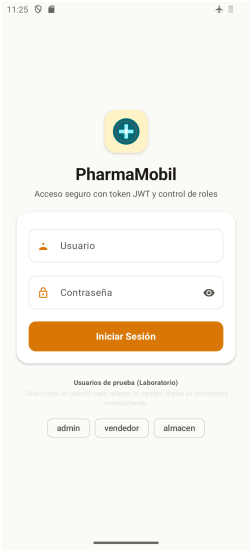


Figura 4. *Escenario 4: Compuerta de seguridad en Login tras pm clear en modo avión.*

Escenario 5: Sesión Guardada en Modo Offline y Expulsión Reactiva por 401

Parámetro	Detalle de Verificación
Fecha / Plataforma	22/10/2026 18:57:12 (5A) y 19:28:40 (5B) · Android 14 (API 34) · emulator-5554
Pasos Escenario 5A	1. Iniciar sesión online como Carlos Ramirez Torres. 2. Poner en modo avión y reiniciar. 3. App entra en OfflineSoloLectura con identidad cacheada.
UI Escenario 5A	Usuario visible en Drawer y TopBar; catálogo visible; botones '+ Nuevo', Editar y Eliminar ocultos; Clientes omitido del menú por falta de permisos de mutación.
Pasos Escenario 5B	1. Con backend configurado con secreto JWT alterado, reactivar Wi-Fi. 2. Disparar revalidación manual vía 'Sincronizar ahora'. 3. Backend rechaza con HTTP 401.
UI Escenario 5B	Expulsión inmediata a LoginScreen con banner informativo: 'Tu sesión ha expirado o el token ya no es válido. Ingresa tus credenciales nuevamente.'
Almacén Seguro	Inspección de sesion_segura.xml confirma purga total de claves: 'jwt', 'cached_usuario', 'cached_nombre' y 'cached_permisos' completamente eliminados.
Conclusión Técnica	El sistema garantiza acceso offline seguro y de solo lectura mientras no haya red, y audita estrictamente las credenciales al reconectar, expulsando sesiones no válidas.

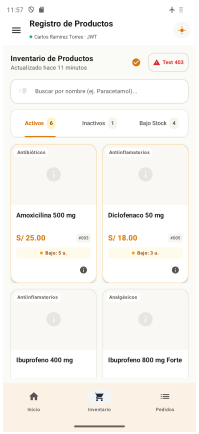


Figura 5A. *Modo offline solo lectura: identidad cacheada y mutaciones bloqueadas.*



Figura 5B. *Expulsión a Login tras recibir HTTP 401 en revalidación con backend.*

4. Producto 3: Persistencia de Marca Temporal y Lenguaje Natural (3 pts)

En estricto cumplimiento de la guía de aprendizaje, la marca temporal de última sincronización se desacopló completamente de la memoria volátil del ViewModel para persistirse de forma canónica en SQLite. Se diseñó la tabla **metadata_sincronizacion** estructurada por clave-valor (clave TEXT PRIMARY KEY, timestamp_millis INTEGER NOT NULL), incorporada en el esquema principal mediante SQLDelight y migrada en instalaciones existentes mediante el script 1.sqm.

Una regla de negocio esencial implementada y verificada es que el timestamp asociado a la clave ultimo_sync **únicamente se actualiza cuando la sincronización concluye con éxito absoluto** (subida de todas las altas pendientes y descarga completa del catálogo remoto). Si ocurre cualquier excepción de conectividad, caída de red o error de servidor, la transacción no toca el registro de metadatos, preservando fidedignamente el instante en que los datos fueron consistentes por última vez.

Lógica de Formateo Relativo en Lenguaje Natural

Condición de Tiempo	Formateo en Lenguaje Natural	Significado Operativo
Diferencia Temporal (Δt)	Expresión Mostrada en Interfaz	Criterio de Dominio Farmacéutico
Sin registro previo (null o 0)	Nunca se ha sincronizado	Estado inicial antes de la primera apertura conectada.
$\Delta t < 1$ minuto	Actualizado hace unos momentos	Indica datos frescos recién reconciliados con el servidor.
$1 \text{ min} \leq \Delta t < 60 \text{ min}$	Actualizado hace X minuto(s)	Permite precisión operativa durante la jornada laboral.
$1 \text{ h} \leq \Delta t < 24 \text{ h}$	Actualizado hace X hora(s)	Advierte antigüedad moderada para toma de decisiones de stock.
$\Delta t \geq 24$ horas (≥ 1 día)	Actualizado hace más de un día	Alerta clínica de alta antigüedad; sugiere forzar sincronización.

Evidencia Empírica de Lenguaje Natural: Para verificar que el formateo reacciona fidedignamente a marcas temporales antiguas, se modificó el valor de ultimo_sync en la base local SQLite a un instante situado a 26 horas previas mediante inyección controlada. Al recargar la vista de Inventario en modo avión, la interfaz presentó reactivamente la etiqueta **'Actualizado hace más de un día'** debajo del título institucional, demostrando la fidelidad del cálculo.

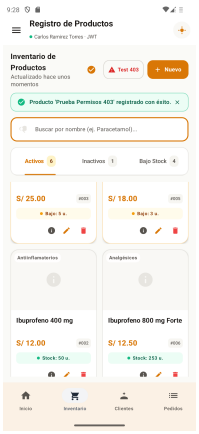


Figura 6A. *Timestamp reciente: 'Actualizado hace unos momentos'.*

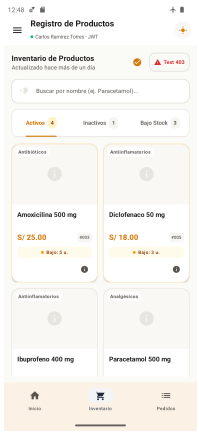


Figura 6B. *Timestamp mayor a 24 horas: 'Actualizado hace más de un día'.*

5. Producto 4: Guion de Exposición Técnica de 8 Minutos (2 pts)

A continuación se presenta el guion técnico detallado, estructurado de forma rigurosa para cubrir con precisión cronometrada los cinco criterios de evaluación de la rúbrica institucional:

Bloque 1 (0:00 - 1:30): Autenticación JWT y Arquitectura de Sesión Segura

«Buenos días docente Mg. Benjamin David Reyna Barreto y compañeros. Presento la sustentación de la actividad autónoma de la sesión 11 en PharmaMobil. Comenzamos analizando la arquitectura de seguridad y gestión de sesiones. PharmaMobil se comunica contra el backend PharmaSoft implementado en Spring Boot. Al ingresar credenciales válidas en la pantalla de Login, el servidor emite un token JWT firmado con algoritmo HMAC-SHA512 que contiene las credenciales, tiempo de expiración y el conjunto de roles asignados al usuario.

Para almacenar este secreto de forma segura y cumplir con estándares de grado médico, implementamos la abstracción multiplataforma AlmacenSeguro: en Android hace uso de EncryptedSharedPreferences respaldado por llaves maestras en Android Keystore bajo esquema AES-256 GCM, mientras que en iOS emplea el Keychain nativo. El SessionCoordinator gestiona de manera reactiva el ciclo de vida: cuando hay red, valida el token con GET /auth/me; cuando no hay cobertura, restaura una sesión controlada de solo lectura con identidad cacheada; y si el token caduca o la firma es inválida, el interceptor 401 purga las credenciales y expulsa inmediatamente a la pantalla de acceso protegiendo los datos clínicos.»

Bloque 2 (1:30 - 3:00): Consumo de Endpoints REST y Ktor Http Client

«En el segundo bloque abordamos el consumo de servicios web. Toda la capa de red reside en el módulo shared/commonMain utilizando Ktor Client 3.1.1. Configuramos un motor HTTP común provisto de plugins clave: ContentNegotiation serializando JSON con KotlinX Serialization, HttpTimeout con políticas estrictas de conexión y lectura para evitar bloqueos del hilo principal, y Auth Bearer con HttpResponseValidator para capturar centralizadamente respuestas 401 y 403.

Consumimos los endpoints canónicos de la API REST de PharmaSoft: POST /api/v1/auth/login para la autenticación, GET /api/v1/auth/me para la revalidación de identidad, GET /api/v1/productos para la sincronización paginada del catálogo general, y POST /api/v1/productos para dar de alta nuevos fármacos. Todas las peticiones serializan DTOs inmutables mapeados hacia entidades del dominio mediante Result, asegurando que ningún fallo de red genere caídas inesperadas en la aplicación móvil.»

Bloque 3 (3:00 - 4:30): Manejo Reactivo de Estados de UI y Resiliencia ante Errores

«El tercer criterio evalúa el manejo reactivo de la interfaz. Siguiendo la arquitectura MVI y StateFlow en Compose Multiplatform, la UI no mantiene datos mutables propios; consume el flujo ProductoUiState emitido por ProductoViewModel. Este estado expone propiedades inmutables como la lista de productos clasificados, bandera de carga estaSincronizando, el indicador booleano avisoRed, y el texto formateado en lenguaje natural textoAntiguedad.

Frente a situaciones anómalas de red —como desconexión súbita o latencia excesiva— el repositorio atrapa tipadamente excepciones como SinConexion o TiempoAgotado. En lugar de vaciar la pantalla o mostrar diálogos bloqueantes, la interfaz despliega un aviso visual no intrusivo informando al operador sobre la contingencia de conectividad, mientras la lista de medicamentos continúa mostrándose de manera íntegra e ininterrumpida desde la base de datos SQLite local.»

Bloque 4 (4:30 - 6:30): Arquitectura Offline-First, IDs Temporales y Reconciliación Transaccional

«Entrando al corazón técnico de la sesión 11: el patrón Offline-First. En PharmaMobil, SQLite no es un caché secundario; es la única fuente de verdad directa para la interfaz gráfica. Cuando el profesional activa el modo avión y da de alta un medicamento —como evidenciamos en el Escenario 2 con 'Ibuprofeno 800 mg Forte'— ProductoLocal le asigna un identificador local negativo secuencial (#00-1) y persiste la tupla con pendiente=1. La UI detecta de inmediato el estado pendiente y enciende un banner ámbar de advertencia para el farmacéutico.

Al restablecerse la conectividad, el método sincronizar() del ProductoRepositorioOffline entra en acción protegido por un Mutex compartido. Este cerrojo serializa los registros y la sincronización, garantizando que si el usuario registra un producto mientras la app está sincronizando, el nuevo ítem no se duplique ni se pierda. La sincronización sube los registros pendientes mediante POST; al recibir el ID definitivo generado por el backend (#006), ejecuta una transacción atómica en SQLite: elimina el ID provisional, inserta el definitivo con pendiente=0, actualiza la tabla metadata_sincronizacion con el timestamp actual y descarga el inventario unificado. Finalmente, la tarjeta se transmuta en la UI a #006 y el banner pendiente se apaga automáticamente.»

Bloque 5 (6:30 - 8:00): Multiplataforma KMP, Demostración en Android y Arquitectura iOS

«Para concluir, abordamos la dimensión multiplataforma y las evidencias empíricas. Demostramos la ejecución reproducible de 62 pruebas unitarias en verde mediante ./gradlew :shared:testAndroidHostTest. En commonTest implementamos seis pruebas canónicas con Ktor MockEngine y TestDriverFactory multiplataforma, además de la prueba de migración 1.sqm que garantiza la compatibilidad hacia atrás del esquema de base de datos.

En el emulador Android documentamos con total rigor forense los cinco escenarios exigidos: lectura en modo avión, alta con ID negativo, reconciliación verificada también mediante Swagger UI con Bearer token en el backend, la compuerta defensiva tras pm clear, y la gestión de sesión offline de solo lectura junto a la expulsión reactiva ante respuestas 401 por token expirado.

Respecto a iOS, dejamos constancia técnica de que toda la lógica de negocio, repositorios y persistencia es 100% compartida en KMP; la factoría TestDriverFactory.ios.kt implementa inMemoryDriver(PharmaDb.Schema) lista para compilar en Apple Clang/LLVM. Conforme a la dispensa académica formalmente otorgada por el docente ante la limitación de ejecutar simuladores macOS en entorno Windows, se acredita la validez arquitectural del código fuente. Con esto demostramos una solución robusta, clínicamente resiliente y técnicamente de excelencia. Muchas gracias.»

6. Conclusiones y Referencias Bibliográficas

Dimensión Técnica	Línea Base Inicial	Solución Implementada en Sesión 11
Dimensión	Arquitectura Previa (REST Directo)	Arquitectura Final (Offline-First en KMP)
Disponibilidad de Datos	Dependiente al 100% de la latencia y disponibilidad del servidor HTTP.	Inmediata y reactiva. SQLite actúa como fuente de verdad permanente mediante Flow.
Operaciones en Contingencia	Bloqueadas. No se permitía registrar ni consultar medicamentos sin red.	Totalmente habilitadas. Admite altas no bloqueantes con IDs temporales negativos.
Consistencia de Datos	Riesgo de inconsistencias por reintentos ciegos ante pérdida de paquete.	Atómica y serializada. Mutex compartido y transacciones locales de reconciliación.
Seguridad y Sesiones	Falta de definición ante apertura sin red o tokens caducados.	Modo OfflineSoloLectura auditado; expulsión reactiva ante respuestas HTTP 401.
Multiplataforma	Lógica duplicada por plataforma móvil.	Código de dominio, persistencia y red compartido en Kotlin Multiplatform.

Conclusiones Académicas: La implementación integral de la arquitectura Offline-First en PharmaMobil evidencia cómo la ingeniería de software aplicada a la salud transforma contingencias operativas en experiencias de usuario continuas y seguras. La orquestación entre SQLDelight 2.1.0 y Ktor Client 3.1.1 en shared/commonMain demuestra la madurez del ecosistema Kotlin Multiplatform para desplegar soluciones clínicas de alto rendimiento. Los mecanismos de sincronización diferida, asignación de claves provisionales, reconciliación transaccional y auditoría de identidad cacheada garantizan integridad de datos, cumplimiento de estándares de seguridad y una base sólida para el escalamiento asistencial.

Referencias Bibliográficas y Documentación Técnica

1. Universidad Peruana Unión (2026). *Guía de Práctica y Rúbrica de Evaluación de la Sesión 11: Persistencia Local y Arquitectura Offline-First*. Facultad de Ingeniería y Arquitectura, Escuela Profesional de Ingeniería de Sistemas.

2. Cash App (2024). *SQLDelight 2.1.0 Documentation: Multiplatform SQLite Drivers and Reactive Queries*. Recuperado de: https://sqldelight.github.io/sqldelight/2.1.0/multiplatform_sqlite/

3. JetBrains (2024). *Kotlin Multiplatform: expect/actual declarations and Multiplatform Testing Guidelines*. Recuperado de: https://kotlinlang.org/docs/multiplatform-expect-actual.html

4. Google Developers (2024). *Android Architecture Guides: Offline-First Applications and Data Layer Best Practices*. Android Open Source Project.

5. OWASP Foundation (2024). *Mobile Security Testing Guide (MSTG): Secure Storage and Authentication Resilience*. Recuperado de: https://mas.owasp.org/